

Phishing URL Detection Using Machine Learning

Stage 2

Data Science for Innovation

Thibaut Barbier · 26456309

Julia Lubelska · 26132415

Tibère Journal · 26456158

Alizée Marchand · 25761139

Table of Contents

1. Literature Review.....	3
1.1 Aljofey et al. (2022): Hybrid URL and HTML features	3
1.2 Hamadouche et al. (2024): Tree ensembles on hybrid features	3
1.3 Almujaheed et al. (2024): Accuracy vs interpretability.....	3
1.4 Fajar et al. (2024): XGBoost with SHAP	3
1.5 Roy and Nilizadeh (2024): LLM-based detection (PhishLang)	3
2. Setup	4
2.1 Aim and research questions.....	4
2.2 Hypotheses.....	4
2.3 Evaluation and reliability.....	4
2.4 Dataset.....	4
3. Approach.....	5
3.1 Exploratory data analysis	5
3.2 Feature engineering	5
3.2.1 Leakage detection.....	5
3.2.2 Feature configurations	5
3.3 Train / validation / test splits	5
3.4 Model training and tuning	5
3.5 Threshold tuning	7
4. Results	8
4.1 Performance on the test set.....	8
4.2 Robustness analysis.....	9
4.3 Interpretability.....	9
4.4 Critical analysis.....	10
5. Conclusion.....	11
5.1 Summary and recommendation.....	11
5.2 Limitations and future work.....	11
5.3 Personal learning.....	11
References	12
Appendix.....	13
Appendix 1 – Figure 1.....	13
Appendix 2- Generative AI Usage Acknowledgement	14

1. Literature Review

Effective phishing URL detection rests on three design choices that motivate our approach: hybrid URL and webpage features, tree-based ensembles for tabular data, and SHAP-based interpretation. The five studies below ground each of these choices in prior empirical evidence.

1.1 Aljofey et al. (2022): Hybrid URL and HTML features

Aljofey et al. [1] integrate URL character sequences, hyperlink information, and HTML statistics, and show that a hybrid feature set outperforms any single category, with accuracy above 96% on 60,252 URLs. URL-only models fail on lexically benign URLs and HTML-only models on sites mimicking legitimate structure; combining them compensates. This justifies retaining both URL features (DomainLength, LetterRatioInURL) and webpage features (IsResponsive, HasPasswordField) in our PhiUSIIL pipeline.

1.2 Hamadouche et al. (2024): Tree ensembles on hybrid features

Hamadouche et al. [2] compare lexical, host-based, and content-based features across Decision Tree, Random Forest, XGBoost, and SVM. The combined feature set yields the highest F1 across every classifier, and Random Forest and XGBoost dominate single-tree and linear baselines (including Logistic Regression), with XGBoost reaching the best F1. This motivates our parallel evaluation of Logistic Regression as an interpretable baseline alongside Random Forest and XGBoost. We also add a domain-based split as a robustness check, since their study did not test generalisation to unseen domains.

1.3 Almujaheed et al. (2024): Accuracy vs interpretability

Almujaheed et al. [3] benchmark Logistic Regression, Random Forest, XGBoost, and a CNN, and report that Random Forest offers a more favourable accuracy-interpretability trade-off than the CNN despite the latter's marginal accuracy gain. Their results also confirm Logistic Regression as a credible transparent benchmark, supporting our decision to retain it alongside the ensembles. This study motivates our choice to exclude a CNN benchmark and to report standardised coefficients for Logistic Regression and permutation importance for the ensembles, since interpretability is operationally critical in security pipelines.

1.4 Fajar et al. (2024): XGBoost with SHAP

Fajar et al. [4] pair gradient-boosted trees with SHAP, showing that XGBoost is efficient on large feature sets and that SHAP reliably isolates the few features driving most of the predictive signal. We follow this approach: SHAP attributions are model-agnostic and additive, providing both global feature ranking and per-instance explanations, and we use them to verify our model is not over-relying on artefactual features such as IsHTTPS, flagged in our EDA as suspiciously deterministic in PhiUSIIL.

1.5 Roy and Nilizadeh (2024): LLM-based detection (PhishLang)

Roy and Nilizadeh [5] introduce PhishLang, a lightweight LLM that detects phishing through contextual analysis of webpage text and identified around 26,000 URLs missed by mainstream blocklists over 3.5 months. LLMs capture semantic patterns that lexical features cannot, but at higher inference cost and lower auditability than tree models with SHAP. A feature-based pipeline remains more appropriate for our research questions, with semantic deception (e.g. brand impersonation) noted as future work.

2. Setup

2.1 Aim and research questions

The aim of this study is to assess whether machine learning models trained on observable URL and webpage characteristics can reliably detect phishing websites, and to identify the configuration (feature set, model family, evaluation split) that offers the best trade-off between predictive performance, robustness to unseen domains, and interpretability. A second aim is to evaluate how much of the measured performance reflects real predictive signal as opposed to dataset-construction effects.

RQ1: Which URL and webpage features are most strongly associated with phishing websites? RQ2: How effectively can ML models detect phishing URLs from observable characteristics? RQ3: Does combining URL and webpage features improve performance over URL-only models, or does the gain mainly reflect dataset-construction artefacts? RQ4: Which model offers the best balance between accuracy, robustness on unseen domains, and interpretability?

2.2 Hypotheses

H0 — Phishing F1-score does not differ across feature configurations or split strategies. H1 — Phishing F1-score differs across configurations and/or splits, indicating that webpage features and/or domain overlap contribute to measured performance. We additionally expect that the F1 gap between Configurations A (URL+Web) and C (URL-strict) will be larger than the gap between stratified and domain-based splits, indicating that feature-level artefacts contribute more to inflated performance than domain memorisation.

2.3 Evaluation and reliability

Phishing is treated as the positive class. Model performance is measured using phishing precision, phishing recall, phishing F1-score, ROC-AUC and PR-AUC. F1 is the main metric because it balances precision and recall, which both matter in phishing detection: false negatives expose users to attacks, while false positives generate alert fatigue. Reliability is supported by reporting every model across three feature configurations and two split strategies (18 experimental settings), which provides a sensitivity range rather than a single point estimate. The Mann-Whitney U test confirmed in Stage 1 that key URL-level distributions differ significantly between classes ($p < 0.001$ for DomainLength and LetterRatioInURL).

2.4 Dataset

The PhiUSIIL Phishing URL Dataset [6] was obtained from the UCI Machine Learning Repository. It contains 235,795 labelled URLs (134,850 legitimate, 100,945 phishing) described by 54 features extracted from URL structure and webpage source code. The dataset has no missing values or duplicates. The class distribution ($\approx 57\%$ legitimate / 43% phishing) is moderately imbalanced, which allows F1, precision and recall to remain informative without resampling. From the original dataset we derived three feature configurations (Section 3.2) and engineered log-transformed versions of highly skewed numerical features for the Logistic Regression pipeline.

3. Approach

3.1 Exploratory data analysis

Stage 1 EDA identified DomainLength, LetterRatioInURL, HTTPS usage, page responsiveness and favicon presence as discriminative signals between phishing and legitimate URLs. Mann-Whitney U tests confirmed significant class differences for the numerical features. However, several features showed near-perfect class separation, suggesting that part of the signal may reflect dataset construction rather than fully generalisable real-world patterns. These suspicious features are explicitly handled in Section 3.2.

3.2 Feature engineering

3.2.1 Leakage detection

Before training, a single-feature AUC was computed for each numerical feature (phishing = positive class). Features with $AUC \geq 0.95$ were considered suspicious. The main issue identified was URLSimilarityIndex, which reached $AUC = 0.996$: it equals 100 for all legitimate URLs and takes a value different from 100 for ~99.2% of phishing URLs, meaning it almost directly reveals the label. It was removed before training. Other features such as LineOfCode, NoOfExternalRef, NoOfImage, NoOfSelfRef, NoOfJS and NoOfCSS also showed very high single-feature AUC values; these were retained but their influence was tested through the ablation described below.

3.2.2 Feature configurations

Three configurations were created. Variables removed unconditionally include URLSimilarityIndex (target leak), IsDomainIP and HasObfuscation (near-zero variance), and highly correlated features such as URLTitleMatchScore, NoOfLettersInURL and NoOfEqualsInURL:

Configuration A — URL + Webpage: all remaining URL, domain and webpage features. Richest set but most exposed to webpage-collection artefacts. Configuration B — URL only: URL and domain-based variables only (length, digit ratio, letter ratio, special-character ratio, HTTPS status, TLD features); no webpage parsing required at inference. Configuration C — URL strict: removes IsHTTPS and TLDLegitimateProb to test whether models can still perform well once likely shortcut variables are excluded.

Configuration	Features (pre-TLD one-hot)
A (URL + Webpage)	61
B (URL only)	18
C (URL strict)	16

Table 1. Feature set configurations.

Configuration A is retained as a diagnostic anchor rather than as a recommended deployment configuration; the gap between A and C is itself the quantitative measure of dataset-construction artefacts. Logistic Regression used log-transformed versions of highly skewed numerical features, while the tree-based models used the raw features.

3.3 Train / validation / test splits

Each configuration was evaluated with two splitting strategies. The first was a stratified 70/15/15 split preserving class proportions. The second was a domain-based split: all URLs from the same domain were kept in a single subset, preventing the model from memorising domain-specific patterns. The domain-based split provides a more realistic estimate of performance on unseen domains. Both splits applied to all three configurations yield six experimental settings.

3.4 Model training and tuning

Three models were compared: Logistic Regression (interpretable benchmark), Random Forest and XGBoost (non-linear, capture feature interactions). Each model $\times$ configuration $\times$ split combination was tuned independently via 3-fold cross-validation with 5 hyperparameter combinations, optimising phishing F1-score (≈ 270 fits in total). Logistic Regression used L2 regularisation with the C parameter tuned via random search; L1 and elastic-net were not explored. Random

Forest was tuned over `n_estimators`, `max_depth`, `min_samples_split` and `max_features`; XGBoost over `n_estimators`, `max_depth`, learning rate and sampling parameters.

3.5 Threshold tuning

After hyperparameter tuning, the classification threshold was selected on the validation set. Thresholds from 0.05 to 0.95 were tested and the value maximising phishing F1 was retained. The selected thresholds differed across models and configurations, confirming that the default 0.50 cutoff is not always optimal.

4. Results

4.1 Performance on the test set

The selected thresholds were applied to the unseen test set (Table 2).

Model type	Feature set	Split type	Threshold	Validation F1-score
logreg	URL+Web	stratified	0.29	1.0000
rf	URL+Web	stratified	0.34	0.9998
xgb	URL+Web	stratified	0.50	1.0000
logreg	URL+Web	domain	0.25	0.9998
rf	URL+Web	domain	0.43	0.9998
xgb	URL+Web	domain	0.86	0.9999
logreg	URL	stratified	0.52	0.9962
rf	URL	stratified	0.64	0.9966
xgb	URL	stratified	0.50	0.9971
logreg	URL	domain	0.40	0.9962
rf	URL	domain	0.64	0.9967
xgb	URL	domain	0.41	0.9976
logreg	URL-strict	stratified	0.32	0.9269
rf	URL-strict	stratified	0.42	0.9889
xgb	URL-strict	stratified	0.40	0.9956
logreg	URL-strict	domain	0.32	0.9218
rf	URL-strict	domain	0.43	0.9876
xgb	URL-strict	domain	0.48	0.9950

Table 2. Final test performance across models, feature sets and splits (phishing = positive class).

The URL+Web configuration delivers near-perfect performance across all models (F1 between 0.9996 and 1.0000); as discussed in Section 4.4, this should be interpreted cautiously. The URL-only configuration also performs strongly (F1 between 0.9959 and 0.9971), indicating that URL structure alone carries substantial predictive signal. In the stricter URL-strict configuration, Logistic Regression drops noticeably to ≈ 0.923 , while Random Forest and XGBoost remain strong; XGBoost achieves the best strict-configuration result (F1 = 0.9943 on the domain split). Figure 1 illustrates the contrast: removing shortcut features sharply increases Logistic Regression's error rate, while XGBoost remains nearly perfect. *Refer to Appendix 1 – Figure 1.*

4.2 Robustness analysis

Robustness was measured as the F1-score difference between stratified and domain-based evaluation. A small drop indicates that the model does not rely on memorising domain-specific patterns. All drops are very small (-0.0002 to 0.0014), with the largest observed for Random Forest URL-strict. The URL+Web models show essentially no drop, but this finding should be qualified by the artefact discussion in Section 4.4. Overall, domain memorisation is not the main driver of measured performance; feature-level effects matter more.

Model type	Feature set	F1 stratified	F1 domain	F1 drop	ROC drop	PR drop
logreg	URL+Web	0.9999	1.0000	-0.0001	0.0000	0.0000
xgb	URL+Web	0.9999	0.9999	0.0000	0.0000	0.0000
rf	URL+Web	0.9996	0.9997	-0.0001	0.0000	0.0000
xgb	URL	0.9969	0.9971	-0.0002	0.0001	0.0000
rf	URL	0.9964	0.9963	0.0001	0.0001	0.0000
logreg	URL	0.9961	0.9959	0.0002	0.0003	0.0001
xgb	URL-strict	0.9950	0.9943	0.0007	0.0001	0.0001
rf	URL-strict	0.9869	0.9856	0.0013	0.0000	0.0000
logreg	URL-strict	0.9235	0.9237	-0.0002	0.0005	0.0006

Table. F1, ROC-AUC and PR-AUC drop between stratified and domain-based evaluation.

4.3 Interpretability

Interpretability analysis was performed on the URL+Web stratified setup. For Logistic Regression, standardised coefficients were examined; for Random Forest and XGBoost, permutation importance on the test set. In all three models, IsHTTPS dominates the ranking (Figure 2). This is expected rather than informative: as flagged in Stage 1, IsHTTPS is deterministic in PhiUSIIL (100% of legitimate URLs use HTTPS), so the models are effectively learning a dataset-specific shortcut. Configuration C was specifically designed to test what the models can learn once this shortcut is removed. Behind IsHTTPS, the top features are mostly webpage-structure variables (LineOfCode, NoOfSelfRef, NoOfExternalRef, NoOfImage) followed by URL-composition signals (LetterRatioInURL, NoOfSubDomain, NoOfOtherSpecialCharsInURL).

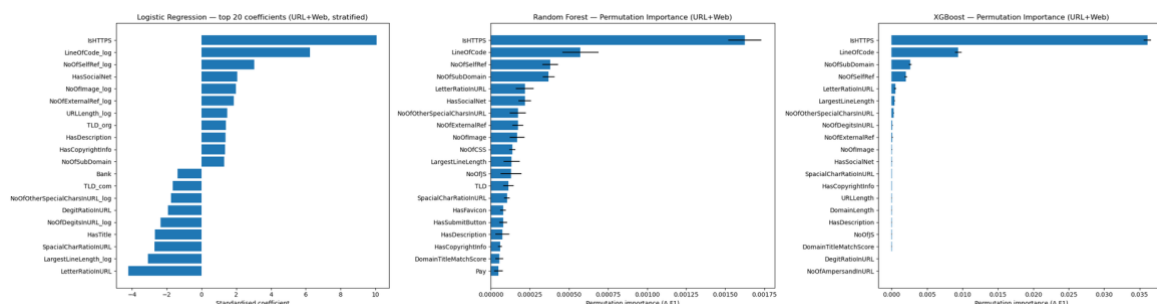


Figure 2. Top-20 feature importance for the three models on URL+Web. IsHTTPS dominates in every model, confirming the shortcut effect.

SHAP analysis on XGBoost (sample of 500 training instances; Figure 3) broadly confirms the permutation-importance ranking. The dominance of webpage-content variables is partly explained by a likely collection artefact in PhiUSIIL (Section 4.4.2), not by an intrinsic property of phishing pages in the wild.

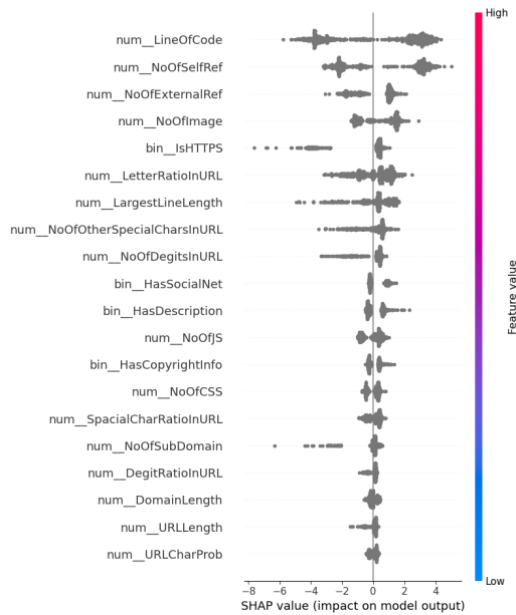


Figure 3. SHAP summary plot for XGBoost (URL+Web, stratified, n=500).

4.4 Critical analysis

The very high F1-scores in the URL+Web configuration require careful interpretation. Three distinct effects amplify performance on this dataset.

Target leakage from URLSimilarityIndex (Section 3.2.1). A single rule on this feature alone reaches $AUC = 0.996$ — strong evidence it was computed using label-adjacent information. Removing it was essential; without this step, all results would be meaningless.

Collection artefact in webpage features. Many phishing examples have near-zero values for LineOfCode, NoOfJS, NoOfCSS, NoOfImage and NoOfExternalRef, while legitimate pages have much larger values. The most plausible explanation is that phishing pages were crawled when already offline, blocked or expired, returning empty or minimal HTML. This makes webpage-complexity features look extremely predictive in PhiUSIIL even though active phishing pages in the wild contain normal HTML/CSS/JS. The gap between Configurations A ($F1 \approx 1.0$) and C ($F1 = 0.92\text{--}0.99$) is the cleanest available estimate of how much of the URL+Web performance is artefactual.

Dataset-construction biases on URL features. 100% of legitimate URLs in PhiUSIIL use HTTPS, and TLDLegitimateProb is itself derived from dataset-wide label statistics. Neither assumption holds in real-world traffic in 2026, where phishing increasingly uses HTTPS via Let's Encrypt. Configuration C removes both. The resulting F1 drop is informative: Logistic Regression loses ~ 7 points ($0.996 \rightarrow 0.923$), while XGBoost loses < 0.3 points ($0.997 \rightarrow 0.994$), indicating that XGBoost recovers most of the signal from structural URL properties whereas Logistic Regression relies on shortcut features.

Taken together, these effects mean $F1 \approx 1.0$ on URL+Web should not be treated as the expected deployment performance. The URL-strict configuration gives the most conservative estimate, and XGBoost still reaches $F1 \approx 0.994$ on the domain split there — a strong, defensible result. The negligible robustness drops in Table 3 further confirm that domain memorisation is not the main inflation source; feature-level artefacts are.

5. Conclusion

5.1 Summary and recommendation

Three findings define the study. (i) All three models reach phishing $F1 \geq 0.92$ even in the most conservative configuration, confirming that ML can detect phishing URLs from observable characteristics. (ii) The apparent superiority of URL+Web ($F1 \approx 1.0$) is largely artefactual, driven by a likely crawl effect and the deterministic HTTPS pattern in PhiUSIIL. (iii) The robustness drop between stratified and domain splits is negligible (< 0.0014 F1 everywhere), so domain memorisation is not the main source of inflated performance — feature-level artefacts are. The diagnostic work in Sections 3.2.1 and 4.4 is therefore at least as important as the model comparison itself.

Two deployment options follow. The first is XGBoost on Configuration B (URL-only): $F1 = 0.9971$ on the domain split, no webpage crawling required, robust to unseen domains — the most appropriate choice for a real-time URL-filtering pipeline. The second is Logistic Regression on Configuration C (URL-strict): it loses ~ 7 F1 points but its standardised coefficients are fully auditable, which matters in security contexts where decisions may need to be explained to analysts, regulators or end users. We do not recommend Configuration A in production despite its $F1 \approx 1.0$, because that performance is partly artefactual and unlikely to hold on a freshly crawled dataset. The choice between the two recommended options depends on whether the organisation prioritises predictive performance or full interpretability.

5.2 Limitations and future work

Results are obtained on a single dataset (PhiUSIIL), so external validity is unconfirmed; webpage features are likely inflated by a collection artefact, and the deterministic HTTPS pattern is not representative of modern phishing; only L2 regularisation was explored for Logistic Regression; and thresholds were tuned for F1 rather than calibrated against operational cost. Future work should target cross-dataset validation (PhishTank, OpenPhish or a freshly crawled corpus), evaluate LLM-based detectors such as PhishLang [5] as a complement for brand-impersonation cases, and develop calibrated cost-sensitive thresholds jointly with security stakeholders.

5.3 Personal learning

The main lesson from this project is that a high F1-score is not, in itself, evidence that a model works. The leakage diagnostic, the configuration-based ablation and the willingness to look behind the headline metric proved at least as valuable as building the models themselves — and, in this case, more important for being able to recommend or not recommend a solution in good faith.

References

- [1] A. Aljofey, Q. Jiang, A. Rasool, H. Chen, W. Liu, Q. Qu, and Y. Wang, "An effective detection approach for phishing websites using URL and HTML features," *Scientific Reports*, vol. 12, p. 8842, 2022.
- [2] K. Hamadouche, R. Saidi, and M. Gasmi, "Combining lexical, host and content-based features for phishing website detection," *Applied Sciences*, vol. 14, no. 19, p. 10086, 2024.
- [3] N. F. Almujaheed, M. A. Haq, and M. Alshehri, "Comparative evaluation of machine learning algorithms for phishing site detection," *PeerJ Computer Science*, vol. 10, p. e1816, 2024.
- [4] A. Fajar, S. Yazid, and I. Budi, "Enhancing phishing detection through feature importance analysis and explainable AI," *arXiv:2411.06860*, 2024.
- [5] S. S. Roy and S. Nilizadeh, "Utilizing large language models to optimize the detection and explainability of phishing websites," *arXiv:2408.05667*, 2024.
- [6] A. Prasad and S. Chandra, "PhiUSILL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning," *Computers & Security*, vol. 136, p. 103545, 2024.

Appendix

Appendix 1 – Figure 1



Figure 1. Selected confusion matrices on the domain split. Left: Logistic Regression collapses on URL-strict (2,289 misclassifications). Centre and right: XGBoost remains accurate both on URL-strict and on URL-only.

Appendix 2- Generative AI Usage Acknowledgement

This report was prepared in accordance with UTS guidelines on the use of generative AI tools. AI assistance was used for IEEE reference formatting, wording suggestions, and consistency review between the literature review, methods and results. All content — including the selection of cited papers, the design of the ablation study, the interpretation of results and the final wording — was produced, reviewed and validated by the authors.